

Strukture podataka i algoritmi

Složenost algoritama

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Timing & counting programa
2. Analiza složenosti algoritama
3. O - Θ - Ω notacije i klase složenosti algoritama

Analiza algoritama

1. Timing & Counting programa

Zašto algoritmi?

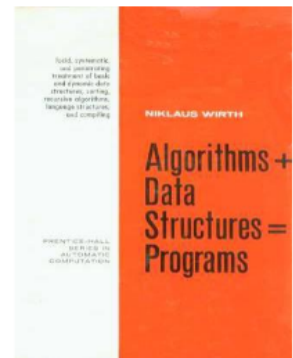
- Da možete da rešavate probleme na pravi način.
- Da postanete vrhunski programer.

“Loš programer postaje dobar programer kada počne da brine i o kodu i o strukturi podataka. Loš programer brine samo o kodu. Dobar programer brine i o strukturi podataka i o njihovim relacijama.”

— Linus Torvalds (tvorac Linux-a)



“Algoritmi + Struktura podataka = Program”
- Niklaus Wirth (tvorac Pascal-a)



Efikasnost programa

- Računari su brzi i postaju sve brži – da li to znači da možda efikasni programi nisu važni?
 - Današnji skupovi podataka mogu biti veoma veliki (procena iz 2024. je da je ukupna količina skladištenih podataka kompanije Google u rasponu od 35–50 eksabajta ili desetine miliona terabajta, a indeksirano je na stotine milijardi veb stranica. **Koliko dugo traje pretraživanje grubom silom?**)
 - Jednostavan algoritam možda neće skalirati veličinu skupova podataka na prihvatljiv način
- Razdvajanje vremenske i prostorne efikasnosti programa
- Kompromis između prostora i vremena (*space-time tradeoff*): može se koristiti više memorije za čuvanje vrednosti i za brže pretraživanje kasnije
 - mogu li se rezultati prethodnih izračunavanja čuvati, a zatim koristiti „pretragu“ za brže dobijanje rezultata (npr. Fibonači niz sa čuvanjem pretrage)
- Fokusirati se na vremensku efikasnost algoritama

Efikasnost programa

- Izazovi u razumevanju efikasnosti rešenja računarskih problema:
 - program se može implementirati na mnogo različitih načina
 - problem se može rešiti koristeći samo nekoliko različitih algoritama
 - želi se razdvajanje izbora implementacije od izbora apstraktnijeg algoritma
- Kako izvršiti evaluaciju efikasnosti programa?
 - merenjem vremena izvršavanja (*timing*)
 - prebrojavanjem izvršenih operacija (*counting*)
 - apstraktni pojam redosleda porasta (*order of growth*)

najprikladniji način procene uticaja
izbora algoritma na rešavanje problema

Vreme izvršavanja programa

```
1  # Merenje vremena konverzije C u F
2
3  import time
4
5  def c_to_f(c):
6      return c*9.0/5 + 32
7
8  tstart= time.time()
9  c_to_f(50)
10 dt = time.time() -tstart
11 print(dt, "s,")
```

time modul

c_to_f(c) funkcija konverzije Celzijus u Farehnajt

Start clock

poziv funkcije

Stop clock

1.430511474609375e-06 s,

c_to_f(1) took 0.0 seconds
c_to_f(10) took 0.0 seconds
c_to_f(100) took 0.0 seconds
c_to_f(1000) took 0.0 seconds
c_to_f(10000) took 0.0 seconds
c_to_f(100000) took 0.0 seconds
c_to_f(1000000) took 0.0 seconds
c_to_f(10000000) took 0.0 seconds

Vreme izvršavanja programa

```
# linear fcn -- finds 0+1+2+...+x
def mysum(x):
    total = 0
    for i in range(x+1):
        total += i
    return total
```



```
mysum(1) took 0.0 sec
mysum(10) took 0.0 sec
mysum(100) took 0.0 sec
mysum(1000) took 0.0 sec
mysum(10000) took 0.0019927024841308594 sec
mysum(100000) took 0.009970903396606445 sec
mysum(1000000) took 0.05089521408081055 sec
mysum(10000000) took 0.4966745376586914 sec
mysum(100000000) took 5.688449382781982 sec
```

Kako se ulaz X
povećava, povećava
se i vreme potrebno
za proračun zbira

Vreme izvršavanja programa

```
# quadratic fcn -- finds n*n inefficiently
def square(n):
    sqsum = 0
    for i in range(n):
        for j in range(n):
            sqsum += 1
    return sqsum
```



```
square(1) took 0.0 sec
square(10) took 0.0 sec
square(100) took 0.0 sec
square(1000) took 0.06244492530822754 sec
square(10000) took 5.553335428237915 sec
```

Kako se ulaz n povećava, povećava se vreme potrebno za proračun $n*n$, za $n=100000$ ne izvršava se u razumnom vremenu

Vremenska evaluacija programa

- **CILJ:** evaluacija različitih algoritama
- vreme izvršavanja varira između algoritama 
- vreme izvršavanja varira između implementacija 
- vreme izvršavanja varira između računara 
- vreme izvršavanja nije predvidljivo za male ulaze 

Vreme izvršavanja algoritma varira za različite opsege ulaza, ali ne može se formulisati stvarna vezu između njih



Broj operacija programa

- Pretpostavka je da konstantno vreme izvršavanja za: matematičke operacije, poređenja, dodele, pristup objektima u memoriji
- Prebrojati broj izvršenih operacija kao funkciju veličine ulaza

c_to_f $\rightarrow$ 3 ops

```
def c_to_f(c):  
    return c*9.0/5 + 32
```

3 ops

mysum $\rightarrow 1+(x+1)*(1+2) = 3x+4$ ops

```
def mysum(x):  
    total = 0  
    for i in range(x+1):  
        total += i  
    return total
```

1 op

loop
x+1 times

2 ops

1 op

square $\rightarrow 1+n*(1)*n*(1+2) = 3n^2 + 1$ ops

```
def square(n):  
    sqsum = 0  
    for i in range(n):  
        for j in range(n):  
            sqsum += 1  
    return sqsum
```

1 op

loop n times

loop n times

1 op

1 op

2 ops

Broj operacija programa

`c_to_f` → 3 ops

```
def c_to_f(c):  
    return c*9.0/5 + 32
```

3 ops



`c_to_f(100)`: 3 ops, 1.0 x more
`c_to_f(1000)`: 3 ops, 1.0 x more
`c_to_f(10000)`: 3 ops, 1.0 x more
`c_to_f(100000)`: 3 ops, 1.0 x more
`c_to_f(1000000)`: 3 ops, 1.0 x more
`c_to_f(10000000)`: 3 ops, 1.0 x more

Bez obzira na
veličinu ulaza, broj
operacija je ISTI

Broj operacija programa

$\text{mysum} \rightarrow 1+(x+1)*(1+2) = 3x+4$ ops

```
def mysum(x):
```

```
    total = 0
```

```
    for i in range(x+1):
```

```
        total += i
```

```
    return total
```

1 op
loop
x+1 times

2 ops

1 op

Ako se ulaz povećava
za 10, broj izvršenih
operacija je približno
10 puta veći

$\downarrow$

```
mysum(100): 304 ops, 1.0 x more  
mysum(1000): 3004 ops, 9.88158 x more  
mysum(10000): 30004 ops, 9.98802 x more  
mysum(100000): 300004 ops, 9.9988 x more  
mysum(1000000): 3000004 ops, 9.99988 x more  
mysum(10000000): 30000004 ops, 9.99999 x more
```

Broj operacija programa

$\text{square} \rightarrow 1+n*(1)*n*(1+2) = 3n^2 + 1 \text{ ops}$

```
def square(n):
```

```
    sqsum = 0
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            sqsum += 1
```

```
    return sqsum
```

square(1): 5 ops, 1.0 x more
square(10): 311 ops, 62.2 x more
square(100): 30101 ops, 96.78778 x more
square(1000): 3001001 ops, 99.69772 x more
square(10000): 300010001 ops, 99.96998 x more

square(128): 49281 ops, 1.0 x more
square(256): 196865 ops, 3.99474 x more
square(512): 786945 ops, 3.99738 x more
square(1024): 3146753 ops, 3.99869 x more
square(2048): 12584961 ops, 3.99935 x more
square(4096): 50335745 ops, 3.99967 x more
square(8192): 201334785 ops, 3.99984 x more

Ako se ulaz povećava
za 10, broj izvršenih
operacija je približno
100 puta veći

Ako se ulaz povećava
za 2, broj izvršenih
operacija je približno
4 puta veći

Broj operacija programa

- **CILJ:** evaluacija različitih algoritama
- Broj operacija zavisi od algoritma 
- Broj operacija zavisi od implementacije 
- Broj operacija ne zavisi od računara 
- Nije precizno definisano koje operacije treba brojati 

Broj operacija algoritma varira za različite ulaze i može se pronaći veza između ulaza i broja operacija

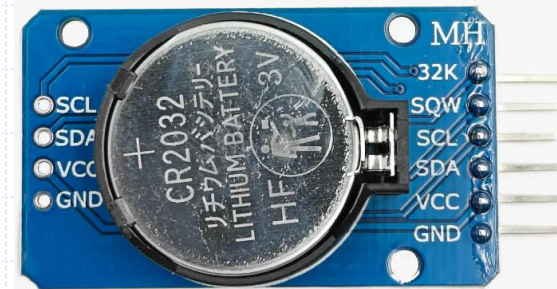
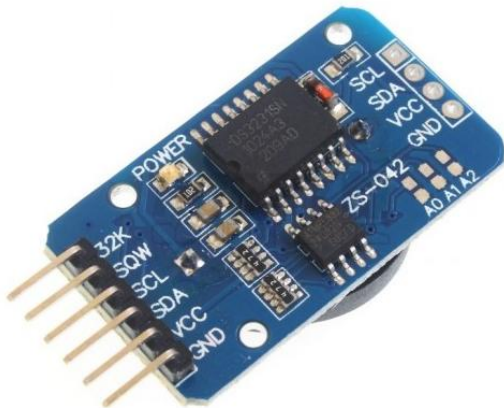


Timing & Counting programa

- Merenje tačnog vremena izvršavanja programa:
 - Zavisi od računara
 - Zavisi od implementacije programa
 - Mali ulazi ne pokazuju porast vremena izvršavanja
- Prebrojavanje broja operacija:
 - Može se dobiti jednačinom!
 - Nezavisno je od računara
 - Zavisi od implementacije programa
 - Multiplikativne/aditivne konstante su irelevantne za velike ulaze
- Potrebno je:
 - Procena (evaluacija) algoritama
 - Evaluacija skalabilnosti
 - Evaluacija u smislu veličine ulaza

Merenje vremena

- Analiza složenosti programa ili delova njegovog kôda, podrazumeva mogućnost merenja proteklog vremena između dva događaja u programskom toku
- Funkciju merenja vremena u računaru vrše dva hardverska časovnika: **sat realnog vremena** i **sistemske sat**
- Sat realnog vremena (RTC): Integrisani čip na matičnoj ploči sistema, poseduje bateriju koja omogućava merenje vremena i kada je računar ugašen, usaglašava se sa preciznim spoljnim časovnicima na Internetu



Prikaz RTC čipa sa baterijom

Merenje vremena

- Sistemska sat: Programabilno tajmersko kolo koje generiše sistemsko vreme (trajanje jedinice sistemskog vremena zavisi od hardverske implementacije i obično je *1ms*, *1μs* ili *1ns*)
- Sistemskim satom vrši se sinhronizovanje (i merenje) raznih procesa u računaru koji traju znatno kraće od onih koji se mere RTC satom
- Po uključanju računara, procesor konvertuje vreme očitano sa RTC čipa u broj vremenskih jedinica proteklih od referentnog datuma u prošlosti
- Tako dobijena vrednost predstavlja sistemsko vreme, a referentni trenutak u prošlosti naziva se *epohom*
- Mnogi operativni sistemi za epohu uzimaju 1. januar 1970, u 00:00:00 po univerzalnom vremenu

Merenje vremena

- Merenje vremena u Pythonu realizuje se pomoću modula `time`
- Funkcije `gmtime()` i `localtime()` vraćaju objekat tipa `struct_time`
- Funkcija `asctime()` prevodi objekat tipa `struct_time` u tekstualnu reprezentaciju pogodnu za ispis
- Vreme proteklo između dva događaja može se odrediti uz pomoć funkcije `time()` koja vraća broj sekundi proteklih od epohe (1. januar 1970, u 00:00:00)

funkcija	opis
<code>gmtime()</code>	vraća objekat koji reprezentuje tekuće univerzalno vreme
<code>localtime()</code>	vraća objekat koji reprezentuje tekuće lokalno vreme
<code>asctime()</code>	vraća tekstualnu reprezentaciju vremenskog objekta
<code>sleep()</code>	pauzira izvršavanje programa za navedeni broj sekundi
<code>time()</code>	vraća broj proteklih sekundi od epohe koja se određuje sa <code>gmtime(0)</code>
<code>perf_counter()</code>	precizno meri trajanje pojedinih delova programa

Merenje vremena

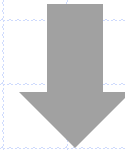
```
>>> import time
>>> time.gmtime() #univerzalno vreme
time.struct_time(tm_year=2025, tm_mon=10, tm_mday=21, tm_hour=15, tm_min=46,
tm_sec=20, tm_wday=1, tm_yday=294, tm_isdst=0)
>>> time.localtime() #lokalno vreme (Bgd)
time.struct_time(tm_year=2025, tm_mon=10, tm_mday=21, tm_hour=17, tm_min=46,
tm_sec=35, tm_wday=1, tm_yday=294, tm_isdst=1)
>>> time.asctime(time.localtime()) #prikaz lokalnog vremena
'Tue Oct 21 17:46:57 2025'
>>> time.asctime(time.gmtime(0)) #prikaz epohe
'Thu Jan  1 00:00:00 1970'
>>> time.time() #br. proteklih sekundi od epohe
1761061737.2380571
>>> |
```

- Vreme proteklo između dva događaja može se odrediti uz pomoć funkcije `time()` koja vraća broj sekundi proteklih od epohe (1. januar 1970, u 00:00:00)
- Funkcija `time()` vraća realan broj, ali njena preciznost nije veća od sekunde

Merenje vremena - primer

- Ako je vreme između dva događaja veoma kratko, koristi se funkcija `perf_counter()` koja precizno meri vreme između dva uzastopna poziva
- Jedan poziv ove funkcije vraća realnu vrednost koja nije vezana za realno vreme, ali se razlika vrednosti iz dva sukcesivna poziva odnosi na trajanje intervala najčešće u `ms` ili `μs`

```
1  # Merenje vremena konverzije C u F
2
3  import time
4
5  def c_to_f(c):
6      return c*9.0/5 + 32
7
8  tstart= time.time()
9  c_to_f(50)
10 dt = time.time() -tstart
11 print(dt, "s,")
```



1.430511474609375e-06 s,

Merenje vremena - primer

Fibonačijev niz

Niz brojeva u kome su prva dva broja 0 i 1, a svaki sledeći je zbir prethodna dva broja:

0, 1, 1, 2, 3, 5, 8, 13, 21,...

$$F(n) := \begin{cases} 0 & \text{ako je } n = 0; \\ 1 & \text{ako je } n = 1; \\ F(n-1) + F(n-2) & \text{ako je } n > 1. \end{cases}$$

```
def fib(n):  
    if n == 0 or n == 1:    #bazni slučaj  
        return n  
    else:                  #rekurzivni korak  
        return fib(n-1) + fib(n-2)
```

```
1 # Poređenje dve rekurzivne verzije za racunanje Fib(n)  
2  
3 import time  
4 # klasican rekurzivni algoritam  
5 def fib(n):  
6     if n == 0 or n == 1:  
7         return 1  
8     else:  
9         return fib(n-1) + fib(n-2)  
10  
11 # rekurzivni algoritam sa memorisanjem  
12 memorija = {}  
13 def fib_memo(n):  
14     if n == 0 or n == 1:  
15         return 1  
16     elif n in memorija:  
17         return memorija[n]  
18     else:  
19         memorija[n] = fib_memo(n-1) + fib_memo(n-2)  
20         return memorija[n]  
21  
22 # unos  
23 n = int(input('Unesite broj n: '))  
24 # bez pamcenja  
25 start = time.perf_counter()  
26 f = fib(n)  
27 t1 = time.perf_counter() - start  
28 print(f'F({n}) = {f} za {t1:10.9f} sekundi')  
29  
30 # sa pamcenjem  
31 start = time.perf_counter()  
32 f = fib_memo(n)  
33 t2 = time.perf_counter() - start  
34 print(f'Fmemo({n}) = {f} za {t2:10.9f} sekundi')  
35  
36 print(f'Memorisanje skraćuje vreme izvršenja {t1/t2:6.1f} puta!')
```

Merenje vremena - primer

```
Unesite broj n: 10
F(10) = 89 za 0.000020700 sekundi
Fmemo(10) = 89 za 0.000013700 sekundi
Memorisanje skraćuje vreme izvršenja 1.5 puta!
```

n=10

```
Unesite broj n: 30
F(30) = 1346269 za 0.141555200 sekundi
Fmemo(30) = 1346269 za 0.000021200 sekundi
Memorisanje skraćuje vreme izvršenja 6677.1 puta!
```

n=30

- Standardni rekurzivni postupak za računanje n-tog člana Fibonačijevog niza realizovan je funkcijom fib() (r5-9); Funkcija fib_memo() (r13-20) koristi rečnik memorija (r12) da upamti prethodno izračunate članove
- Ubrzanje odnosu na klasični pristup postiže se tako što se proverava da li je traženi n-ti član već izračunat (r16), ako jeste funkcija vraća vrednost sačuvanu u rečniku (r17), u suprotnom F(n) se računa pomoću prethodna dva člana, a dobijena vrednost stavlja u rečnik (r19) pa potom vraća kao rezultat rada funkcije (r20)

Skraćivanje vremena izvršavanja je često nauštrb povećanog korišćenja memorijskih resursa

Merenje vremena - primer

Primer: Kôd koji računa zbir prirodnih brojeva od 1 do i za svako i u intervali od 1 do n (dva načina)

```
zbir = 0
for i in range(1, n + 1): # Iteracija od 1 do n

    zbir += i

print(zbir)
```

n = 1000000	rezultat: 500000500000	vreme: 0.16 sekundi
n = 2000000	rezultat: 2000001000000	vreme: 0.31 sekundi
n = 3000000	rezultat: 4500001500000	vreme: 0.49 sekundi
n = 4000000	rezultat: 8000002000000	vreme: 0.65 sekundi
n = 5000000	rezultat: 12500002500000	vreme: 0.83 sekundi

Vremena izvršavanja su približno srazmerna vrednostima za n tj. približno **linearno** zavisi od n

```
def zbir(k):
    total = 0
    for i in range(1, k + 1): # Iteracija od 1 do k

        total += i
    return total

n = int(input("Unesite broj n: "))
for k in range(1, n + 1): # Iteracija od 1 do n

    print(zbir(k))
```

n = 1000	rezultat: 500500	vreme: 31.24 ms
n = 2000	rezultat: 2001000	vreme: 156.25 ms
n = 3000	rezultat: 4501500	vreme: 359.35 ms
n = 4000	rezultat: 8002000	vreme: 633.19 ms
n = 5000	rezultat: 12502500	vreme: 993.25 ms
n = 6000	rezultat: 18003000	vreme: 1448.38 ms
n = 7000	rezultat: 24503500	vreme: 1984.94 ms
n = 8000	rezultat: 32004000	vreme: 2547.53 ms
n = 9000	rezultat: 40504500	vreme: 3291.45 ms
n = 10000	rezultat: 50005000	vreme: 4049.11 ms

Ako se n udvostruči, vreme izvršavanja se uveća $\sim 4\times$, što znači da je srazmerno sa **kvadratom veličine ulaza**; Ako se izračuna vreme $t(n)/n^2$ za razno n , dobija se vrednost od $\sim 0.00004n^2$, pa se vreme izvršavanja može aproksimirati kao **kvadratno** ili $t(n) \approx 0.00004n^2$

Merenje vremena - primer

- Vreme izvršavanja algoritma za veliki broj ulaznih podataka znači da nas ne interesuje apsolutno tačno vreme njegovog izvršavanja, već koliko brzo raste vreme izvršavanja sa povećanjem veličine ulaznih podataka
- Da bimo procenili zavisnost vremena izvršavanja od dimenzije problema, osnovni pristup je da formiramo funkciju $f(n)$ koja određuje zavisnost broja instrukcija koje algoritam treba da izvrši u odnosu na veličinu ulaza n
- U prvom programu se vrši jedno sabiranje po petlji ili iteraciji, pa je ukupan broj sabiranja jednak broju koraka izvršavanja petlje (a to je n) i to je **linearna zavisnost**
- U drugom programu za bilo koje dato k , funkcija **zbir** vrši približno k sabiranja; Pošto se funkcija poziva za sve vrednosti k od 1 do n , ukupno će se izvršiti $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$ sabiranja, pa je broj instrukcija kvadratna funkcija oblika $an^2 + bn + c$

Analiza algoritama

2. Analiza složenosti algoritama

Složenost algoritama

- **Složenost algoritama** je vrlo važna tema u računarstvu koja se bavi analizom koliko resursa (vremena i memorije itd.) algoritam troši u odnosu na veličinu ulaznih podataka
- Ova vrsta analize omogućava da se razume koliko je algoritam efikasan, posebno kada obrađuje velike skupove podataka
- Ovaj koncept je kritičan za optimizaciju i sprečavanje problema sa performansama, posebno u velikim i složenim softverskim projektima
- Algoritmi visoke složenosti mogu dovesti do ozbiljnih problema sa performansama kada se koriste za velike skupove podataka
- Izbor algoritma i njegova optimizacija je pitanje koje se mora stalno razmatrati u procesu razvoja softvera

Složenost algoritama

- Cilj analize složenosti je da utvrdi ponašanje algoritma za različite kombinacije ulaznih veličina kako bi se omogućilo poređenje sa drugim rešenjima, u smislu utroška vremenskih i operativnih resursa
- Pored svojstva ispravnosti programa, veoma je važno i pitanje koliko program zahteva vremena (ili izvršenih instrukcija) i memorije za svoje izvršavanje
- Mnogi problemi mogu se rešavati na puno načina i nije svako rešenje jednako dobro
- Najbolja rešenja podrazumevaju minimalnu potrošnju resursa, pa se uglavnom razmatraju:
 - 1) vremenska složenost algoritma**
 - 2) prostorna (memorijska) složenost algoritama**

Procena vremena izvršavanja programa

- Vreme izvršavanja pojedinih delova kôda (na konkretnom računaru) može da se proceni ukoliko je raspoloživa procena vremena izvršavanja pojedinih operacija
- Primer: na PC računaru sa procesorom Intel Core i7-2670QM na 2.2GHz koji radi pod Linux OS, za operaciju sabiranja dve vrednosti tipa `int` potrebno je $\sim 0,3ns$ a za operaciju množenja $\sim 1ns$
- Ova procenu treba uzimati sa rezervom, jer ona ne podrazumeva neke bitne aspekte izvršavanja programa (koji se još procesi trenutno izvršavaju, koji je operativni sistem, kako je napravljena izvršna verzija programa itd)

Profajleri

- Procenu vremena izvršavanja pojedinih operacija i kontrolnih struktura na računaru moguće je izmeriti
- Postoje mnogi alati za analizu i unapređenje performansi programa i nazivaju se **profajleri** (*profiler*), a njihova osnovna uloga je da daju podatke o tome koliko puta je koja funkcija pozvana tokom (nekog konkretnog) izvršavanja, koliko je utrošila sistemskog vremena itd.
- Ukoliko se razvijeni programski kôd ne izvršava dovoljno brzo, prvi kandidati za izmenu su delovi kôda koji troše najviše vremena
- Primeri Python profajlera: **cProfile**, **line_profiler**, **memory_profiler**, **Scalene**, **Pyinstrument**, **py-spy** itd.

Funkcija složenosti algoritma

- **Funkcija složenosti** algoritma je analitička funkcija koja povezuje veličinu problema (ulaznih podataka) i vremena izvršavanja algoritma ili utrošenih memorijskih reursa
- Vremenska i prostorna složenost mogu se razmatrati:
 - u okvirima konkretnog vremena ili prostora utrošenog za konkretne ulazne podatke
 - u okvirima asimptotskog ponašanja vremena ili prostora kada se količina ulaznih podataka povećava
- Vremenska i prostorna složenost algoritma određuje i njegovu praktičnu upotrebljivost tj. najveću dimenziju ulaza za koju je moguće da će se algoritam izvršiti u nekom razumnom vremenu ili koristeći postojeće resurse

Funkcija vremenske složenosti

```
1 # Poređenje dva algoritma za sumiranje prvih n prir. brojeva
```

```
2  
3 from time import perf_counter
```

```
4  
5 def sumaA(n): # Algoritam A  
6     return n * (n + 1) // 2
```

```
7  
8 def sumaB(n): # Algoritam B  
9     suma = 0  
10    for i in range(1, n+1):  
11        suma += i  
12    return suma
```

```
13  
14 # test program za A
```

```
15 for n in [100, 1000, 10_000, 100_000, 1_000_000]:  
16     start = perf_counter()  
17     s = sumaA(n)  
18     t = perf_counter() - start  
19     print(f'A: n={n:9} s={s:13} za {t:10.9f} sec')
```

```
20  
21 # test program za B
```

```
22 for n in [100, 1000, 10_000, 100_000, 1_000_000]:  
23     start = perf_counter()  
24     s = sumaB(n)  
25     t = perf_counter() - start  
26     print(f'B: n={n:9} s={s:13} za {t:10.9f} sec')
```

A: n=	100	s=	5050	za	0.000001400	sec
A: n=	1000	s=	500500	za	0.000003700	sec
A: n=	10000	s=	50005000	za	0.000001200	sec
A: n=	100000	s=	5000050000	za	0.000001100	sec
A: n=	1000000	s=	500000500000	za	0.000000500	sec
B: n=	100	s=	5050	za	0.000007000	sec
B: n=	1000	s=	500500	za	0.000030500	sec
B: n=	10000	s=	50005000	za	0.000303300	sec
B: n=	100000	s=	5000050000	za	0.003363000	sec
B: n=	1000000	s=	500000500000	za	0.033912200	sec

Funkcija vremenske složenosti

- Porede se vremena izvršavanja dva algoritma A i B, za izračunavanje sume prvih n prirodnih brojeva
- Vreme izvršavanja algoritma A je približno konstantno i ne zavisi od n
- Algoritam B je sporiji, ako se n poveća $10\times$, njegovo vreme izvršavanja se poveća $\sim 10\times$ (vreme izvrš. algoritma B **linearno raste** sa porastom broja n)
- $t(n)$ je funkcija vremenske složenosti
- Funkcija `sumaA()` sastoji se od programskog reda (r6), izračunavanje i vraćanje izraza vrši se za const. vreme c , pa je $t_A(n) = c$
- Funkcija složenosti za `sumaB()` može se izraziti kao

$$t_B(n) = c_1 + n(c_2 + c_3) + c_4 = an + b$$

C1 – dodela vrednosti (r9)

C2 – izvršavanje naredbe petlje (r10), n puta

C3 – sumiranje (r11), n puta

C4 – naredba return (r12)

Funkcija vremenske složenosti

- Za algoritme A i B porede se njihove funkcije složenosti $t_A(n)$ i $t_B(n)$ i porede se brzine rasta ove dve funkcije sa porastom ulaza n

Efikasniji je onaj algoritam čija funkcija složenosti sporije raste u opštem slučaju, za proizvoljno veliko n

- Za algoritme A i B važi:
$$t_B(n) > t_A(n) \text{ za } n > \frac{c-b}{a}.$$
- Univerzalni pristup u analizi funkcije složenosti koji neće zavisiti od hardverskih i softverskih karakteristika cilnog sistema podrazumeva sledeća pojednostavljenja:
 - trajanje izvršavanja algoritma za određenu dimenziju problema treba izraziti preko broja dominantnih operacija u algoritmu
 - funkciju složenosti treba posmatrati u graničnom slučaju kada brojnost ulaznih podataka teži beskonačnosti

Analiza slučajeve funkcije složenosti

- Algoritmi u praksi imaju kompleksne ulazne podatke sa različitim raspodelama verovatnoća
- Zato se funkcije složenosti mogu posmatrati za 3 različita slučaja:
 - Najpovoljniji slučaj
 - Prosečan slučaj
 - Najnepovoljniji ili najgori slučaj!!!

```
def is_in(L, e):  
    for i in L:  
        if i == e:  
            return True  
    return False
```

Primer: proverava da li se karakter **e** nalazi u listi tako što se kroz nju prolazi element po element

- Kada je **e** na početku liste → **BEST CASE**
- Kada je **e** na sredini liste → **AVERAGE CASE**
- Kada **e** nije u listi → **WORST CASE**

Redovi rasta funkcije složenosti

- Najnepovoljniji slučaj određuje krajnji domet algoritma, što je od velikog značaja za njegovu praktičnu primenu
- Da bi se prilikom poređenja dva algoritma zanemarili efekti hardverske konfiguracije računara, porede se **redovi rasta njihovih funkcija složenosti** (*orders of growth*)
- Posmatra se količnik funkcija složenosti 2 računara (A se izvršava na brzom, a B na sporom računaru) **u asimptotskom slučaju** kada $n \rightarrow \infty$

$$\lim_{n \rightarrow \infty} \frac{t_B(n)}{t_A(n)} = \frac{dn + e}{an^2 + bn + c} = 0$$

- Iako se B izvršava na sporijem računaru, njegova efikasnost u odnosu na A u najnepovoljnijem slučaju je značajno veća jer za veliko n u $t_A(n)$ dominira kvadratni član!

„Dobar algoritam i spor računar bolja je opcija od lošeg algoritma i brzog računara“

Analiza slučajeva funkcije složenosti

- Najčešće se algoritmi ne izvršavaju isto za sve ulaze istih veličina
 - Analiza najgoreg slučaja vrši procenu složenosti algoritma za najgori slučaj (slučaj da se algoritam najduže izvršava ili troši najviše memorije) i daje jake garancije da program koji je u najgorem slučaju dovoljno efikasan, u svim slučajevima može da se izvrši sa raspoloživim resursima
 - Nekada je moguće izračunati prosečno vreme izvršavanja algoritma, ali je za to potrebno precizno poznavanje opsega ulaznih vrednosti i verovatnoće pojave tih vrednosti; Procena prosečnog izvršavanja može biti varljiva i previše optimistična
 - Analiza najboljeg slučaja je previše optimistična i nema smisla
- Nekada se analiza vrši tako što se procenjuje ukupno vreme potrebno da se izvrši određeni broj srodnih operacija, i to se naziva **amortizovana analiza** (bitno je zbirno vreme izvršavanja svih operacija)

Analiza algoritama

3. O - Θ - Ω notacije i klase složenosti algoritama

Asimptotska analiza algoritama

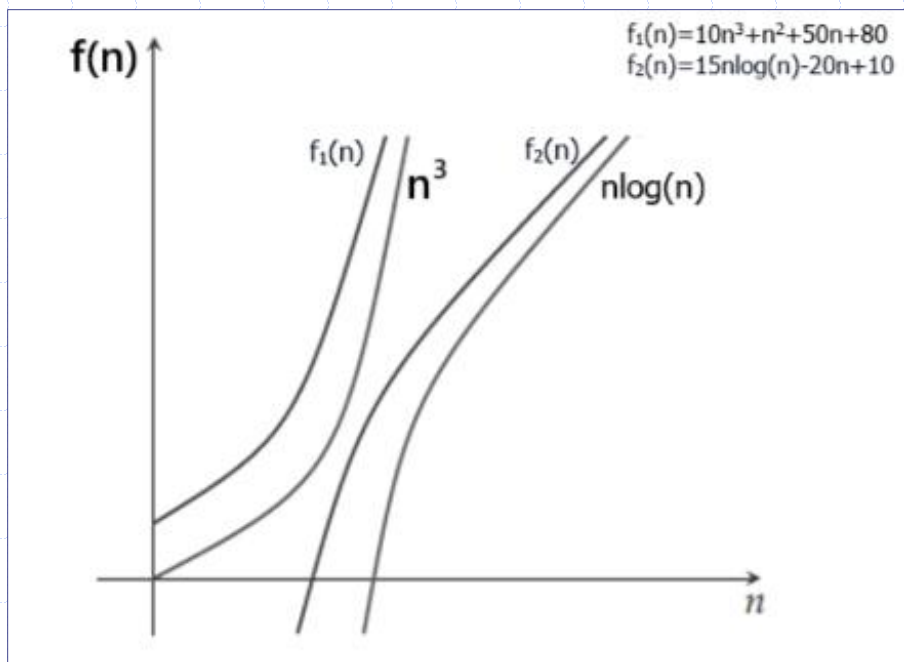
- Tačno vreme izvršavanja algoritama je unapred skoro nemoguće izračunati pa se vrši njihova procena ili evaluacija
- Usled kompleksnosti ili nemogućnosti izvođenja egzaktne formule za broj operacija nekog algoritma, koristi se asimptotsko ponašanje broja operacija kad veličina ulaznih podataka neograničeno raste
- Analizu algoritma pojednostavljujemo i tražimo red veličine funkcije vremena njegovog izvršavanja za veliki broj ulaznih elemenata
- Približno vreme izvršavanja naziva se asimptotsko vreme izvršavanja algoritma i predstavlja meru brzine rasta tačnog vremena izvršavanja algoritma sa povećanjem broja njegovih ulaznih podataka
- Za opis vremenske složenosti koristi se termin asimptotska notacija
- Član funkcije vremena izvršavanja sa najvećim eksponentom (vodeći član) najviše utiče na njen rast pa je on najvažniji za opis vremenske složenosti

Asimptotska analiza algoritama

- **Cilj:** Analiza povećanja vremena izvršavanja algoritma sa povećanjem veličine ulaza
- Metoda koja povezuje veličinu ulaza sa brojem operacija
- Zna se izraz za broj operacija potrebnih za izvršavanje algoritma, želi se znati asimptotsko ponašanje kako veličina problema raste
 - Utvrđivanje granica rasta
 - Ne koristi se precizan proračun: reč „red rasta“ a ne „tačan rast“
- Fokus je na članu koji najbrže raste
- Zanimaju se aditivne i multiplikativne konstante, jer se želi znati koliko brzo se vreme izvršavanja povećava sa povećanjem veličine ulaza
- Naziv: red rasta (*order of growth*)
- Koriste se matematički pojmovi „veliko O“ i „veliko TH“

Asimptotska analiza algoritama

- Pojednostavljenje proračuna vremena izvršavanja algoritma podrazumeva:
 - Vreme izvršavanja algoritama značajno se povećava sa povećanjem broja ulaznih podataka
 - Za dovoljno veliku vrednost n , funkcija vremena izvršavanja $f(n)$ je određena vodećim članom izraza funkcije
 - Jednostavna funkcija vremena izvršavanja uzima se bez konstanti, koje su u odnosu na vodeći član najčešće beznačajne



- Krive $f_1(n)$ i n^3 , kao i $f_2(n)$ i $n \log(n)$ su vrlo blizu za velike vrednosti n
- Za velike vrednosti n može se smatrati da su ove krive jednake
- Asimptotsko vreme izvršavanja algoritama može se napisati izrazima

$$f_1(n) \approx n^3 \text{ i } f_2(n) \approx n \log(n)$$

O-notacija

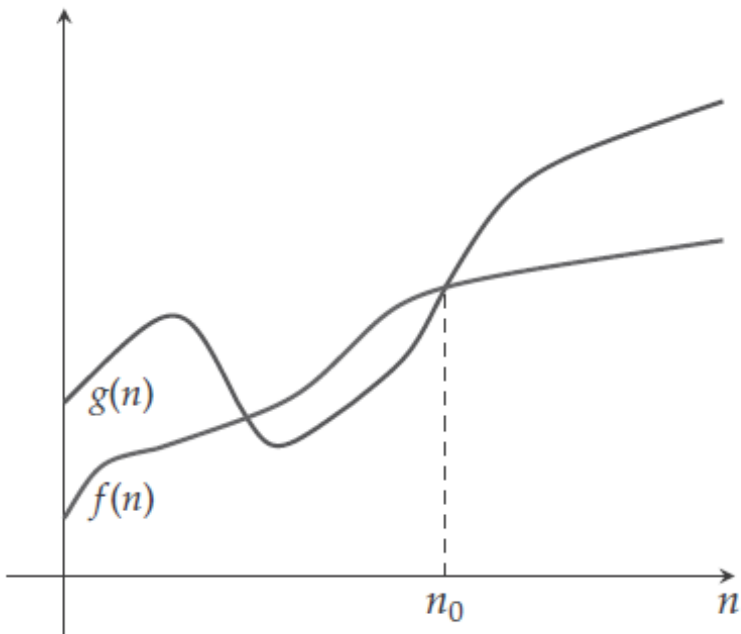
- O-notacija (Edmund Landau, Paul Bachman, početak XX veka) definiše procenu vremensko/prostornih resursa algoritama
- Matematička notacija „Big O“
- Predstavlja granično ponašanje funkcije kada argument teži nekoj specifičnoj vrednosti ili beskonačnosti
- Definiše se preko jednostavnijih funkcija
- Gornja granica složenosti algoritma se izražava O-notacijom:

Ako postoje pozitivna realna konstanta c i prirodan broj N takvi da za funkcije f i g nad prirodnim brojevima važi $f(n) \leq c \cdot g(n)$ za sve prirodne brojeve $n > N$ onda pišemo $f = O(g)$ i čitamo "f je veliko 'o' od g"

- "O" ne označava konkretnu funkciju, već klasu funkcija i uobičajeni zapis $f = O(g)$ zapravo znači $f \in O(g)$

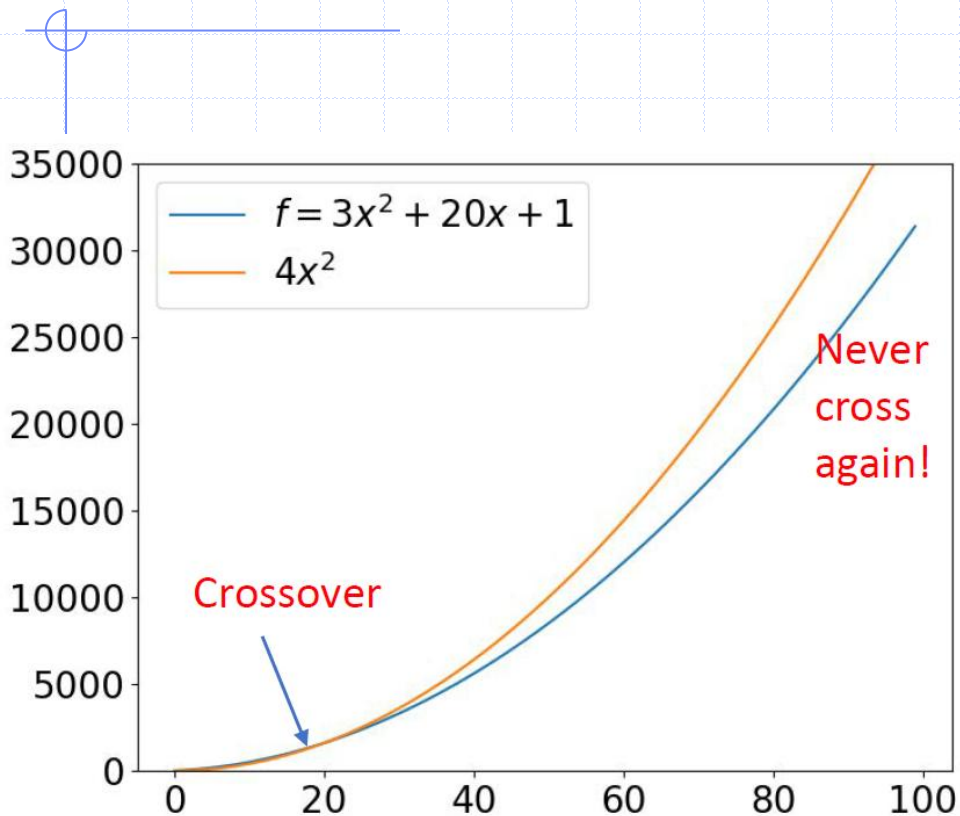
O-notacija

- O-notacija se koristi za precizno defnisanje pojma da je neka funkcija „manja“ od druge
- Izrazom $f(n)=O(g(n))$ se opisuje svojstvo da je funkcija f manja od funkcije g , odnosno da g dominira nad f



- $f(n)=O(g(n))$ znači da postoji tačka n_0 takva da za sve vrednosti n od te tačke, funkcija $f(n)$ je ograničena vrednošću proizvoda pozitivne konstante c i funkcije $g(n)$
- Funkcija $g(n)$ je asimptotska gornja granica za funkciju $f(n)$
- Da bismo za konkretne funkcije pokazali da je vreme izvršavanja algoritma $T(n)=O(g(n))$ potrebno je naći n_0 i $c>0$ tako da je $T(n) \leq cg(n)$ za svako $n \geq n_0$

O-notacija



$$4x^2 > 3x^2 + 20x + 1 \forall x > 20.04$$

$$3x^2 + 20x + 1 = O(x^2)$$

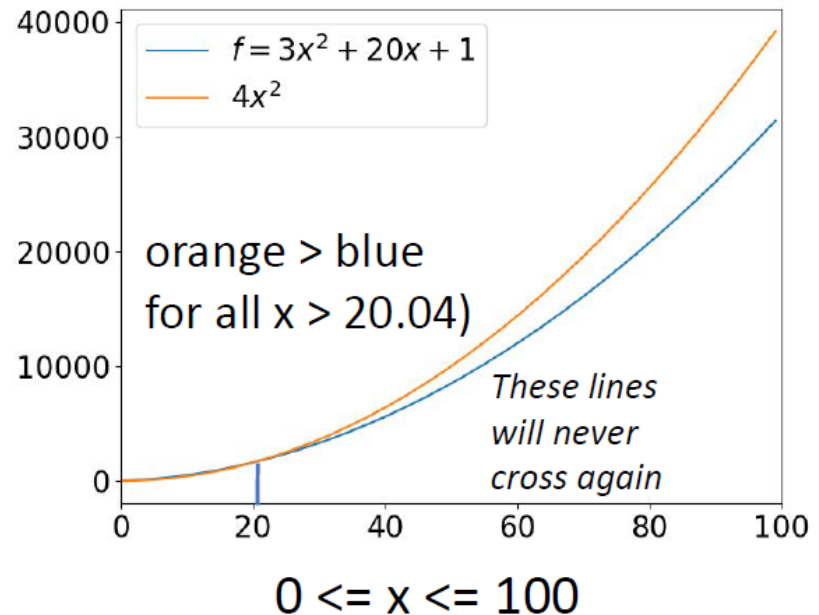
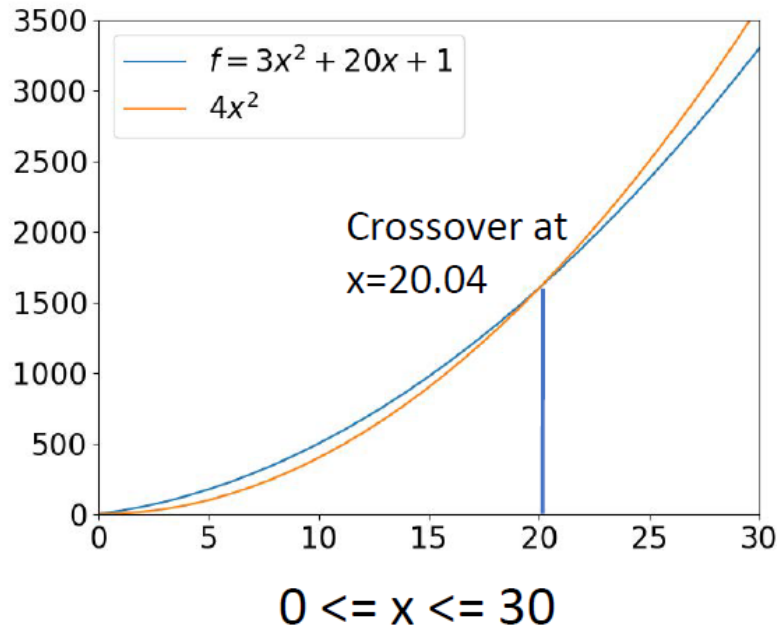
- Pretpostavka da se kôd izvršava u $f(x) = 3x^2 + 20x + 1$ koraka
- To je izraz za counting broja operacija
- Veliko O je način da se ograniči rast funkcije
- **$f(x) = O(g(x))$** znači da $g(x)$ pomnoženo sa nekom konstantom na kraju uvek prelazi $f(x)$
- „Na kraju” znači iznad neke granične vrednosti x

O-notacija (formalno)

- "Veliko O" je gornja granica rasta neke funkcije
- Primer: $f(x) = 3x^2 + 20x + 1$

$$f(x) = O(x^2), \text{ because } 4x^2 > 3x^2 + 20x + 1 \forall x \geq 21$$

$(c_0 = 4, x_0 = 20.04)$



O-notacija

- Zanemariti aditivne i multiplikativne konstante
- Glavni su članovi funkcije najvišeg stepena

$$O(n^2) : n^2 + 2n + 2$$

$$O(n^2) : n^2 + 1000000n + 3^{1000}$$

$$O(n) : \log(n) + n + 4$$

$$O(n \log n) : 0.0001 * n * \log(n) + 300n$$

$$O(3^n) : 2n^{30} + 3^n$$

O-notacija

- Zanemariti aditivne i multiplikativne konstante
- Glavni su članovi funkcije najvišeg stepena

$$\Theta(x) : 1000 * \log(x) + x$$

$$\Theta(n^3) : n^2 \log(n) + n^3$$

$$\Theta(y) : \log(y) + 0.000001y$$

$$\Theta(2^b) : 2^b + 1000a^2 + 100 * b^2 + 0.0001a^3$$

$$\Theta(a^3)$$

$$\Theta(2^b + a^3)$$

O-notacija

- Primer: U izrazu ($5n^2 + 1$) konstanta 1 je aditivna, a konstanta 5 je multiplikativna i ove konstante ne utiču na klasu kojoj funkcija pripada
- Zato se kaže da je neki algoritam složenosti $O(n^2)$ a ne $O(5n^2+1)$ jer aditivna konstanta 1 i multiplikativna 5 nisu od suštinske važnosti
- Ako jedan algoritam zahteva $5n^2+1$ instrukcija, a drugi n^2 , i ako se prvi algoritam izvršava na računaru koji je 6 puta brži od drugog, on će biti brže izvršen za svaku veličinu ulaza !!!
- Ako jedan algoritam zahteva n^2 instrukcija, a drugi n , ne postoji računar na kojem će prvi algoritam da se izvršava brže od drugog za svaku veličinu ulaza !!!

O-notacija (primeri i svojstva)

$$n^2 = O(n^2)$$

$$n^2 + 10 = O(n^2)$$

$$10 \cdot n^2 + 10 = O(n^2)$$

$$10 \cdot n^2 + 8n + 10 = O(n^2)$$

$$n^2 = O(n^3)$$

$$2^n = O(2^n)$$

$$2^n + 10 = O(2^n)$$

$$10 \cdot 2^n + 10 = O(2^n)$$

$$2^n + n^2 = O(2^n)$$

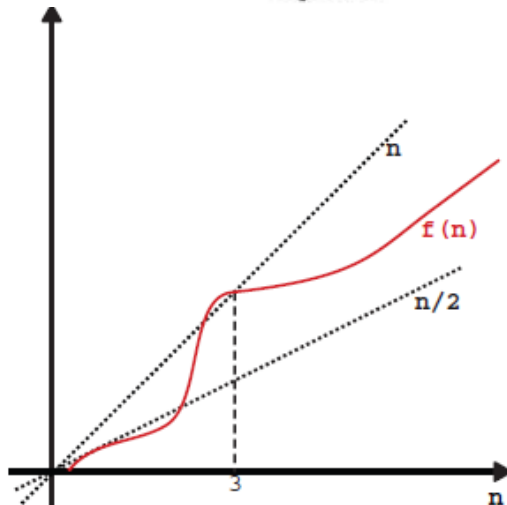
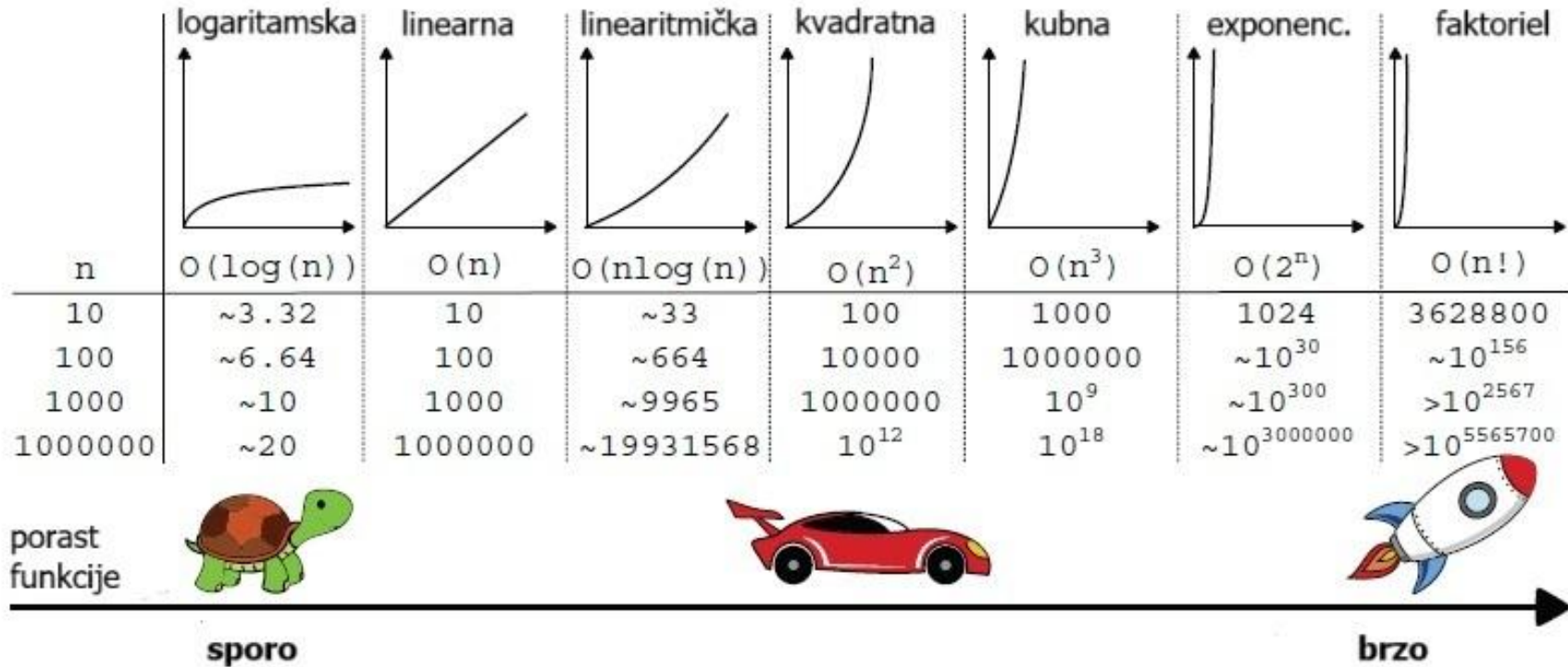
$$3^n + 2^n = O(3^n)$$

$$2^n + 2^n n = O(2^n n)$$

Za O-notaciju veže sledeća svojstva:

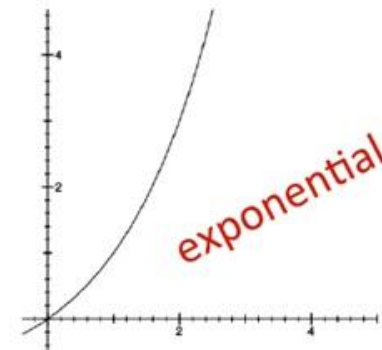
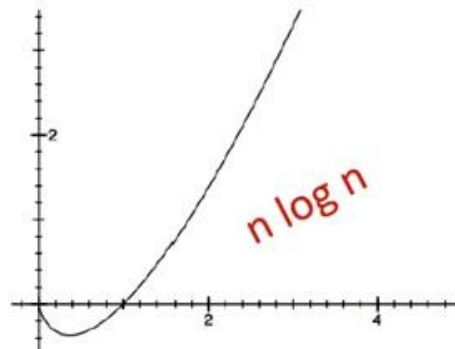
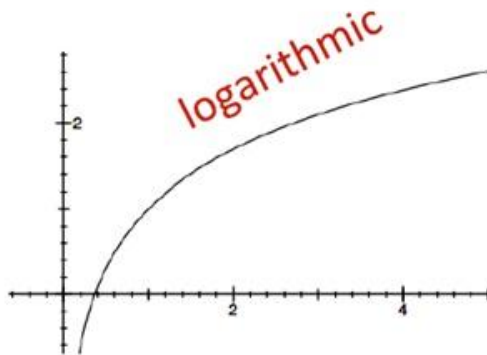
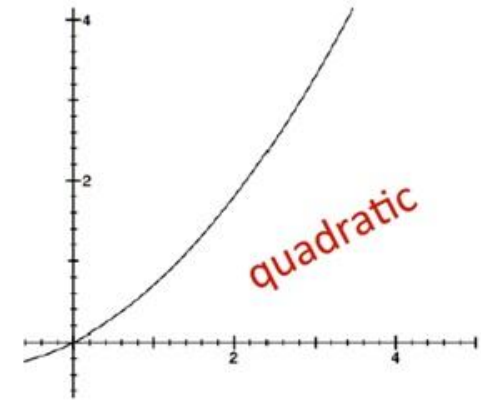
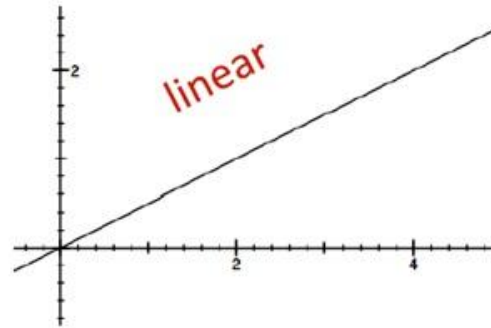
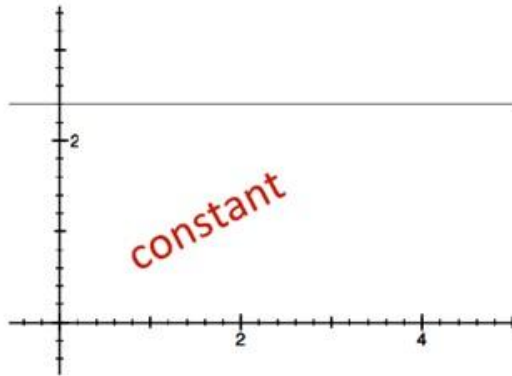
- ako važi $f_1 = O(g_1)$ i $f_2 = O(g_2)$ onda važi
$$f_1 \cdot f_2 = O(g_1 \cdot g_2)$$
$$f_1 + f_2 = O(g_1 + g_2)$$
- ako važi $f = O(g)$ i a i b su pozitivne konstante, onda važi
$$af + b = O(g)$$

Klase složenosti



O-notacija se koristi za precizno defnisanje pojma da je neka funkcija „manja“ od druge, ili da je u granicama između dve funkcije

Klase složenosti

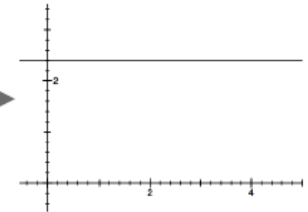


Klase složenosti

$O(1)$

:

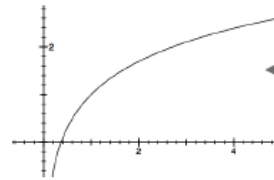
constant



$O(\log n)$

:

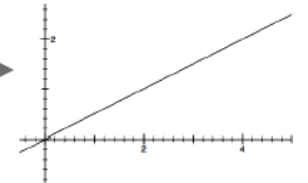
← logarithmic



$O(n)$

:

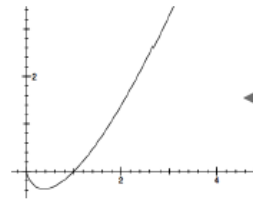
linear



$O(n \log n)$

:

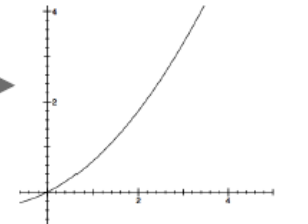
← loglinear



$O(n^c)$

:

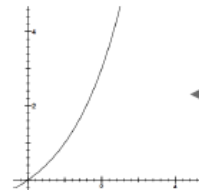
polynomial



$O(c^n)$

:

← exponential



*c is a
constant*

Analiza složenosti programa

- Kombinovanje klasa složenosti:
 - analiza iskaza unutar funkcija
 - primena nekih pravila, fokus na dominantnom članu
- Zakon sabiranja/množenja za $O()$:
 - za sekvencijalne iskaze
 - Za ugneždene iskaze/petlje
 - $O(f(n)) + O(g(n))$ je $O(f(n) + g(n))$
 - $O(f(n)) * O(g(n))$ je $O(f(n) * g(n))$

```
for i in range(n):  
    print('a')  
for j in range(n*n):  
    print('b')
```

$O(n)$

$O(n*n)$



$O(n) + O(n*n)$

$$O(n) + O(n*n) = O(n+n^2) = O(n^2)$$

```
for i in range(n):  
    for j in range(n):  
        print('a')
```

$O(n)$



n loops, each $O(n) \rightarrow O(n)*O(n)$

$$O(n)*O(n) = O(n*n) = O(n^2)$$

Analiza složenosti programa

- Šta je Θ -složenost?

Always note the input parameter!

```
def f(x):
```

```
    answer = 1
```

```
    for i in range(x):
```

```
        for j in range(i, x):
```

```
            answer += 2
```

```
    return answer
```

Outer loop is $\Theta(x)$

Inner loop is $\Theta(x)$

Everything else is $\Theta(1)$

$$\Theta(1) + \Theta(x) * \Theta(x) * \Theta(1) + \Theta(1)$$

- Sveukupna složenost je $\Theta(x^2)$ zbog pravila sabiranja i množenja

Klase složenosti

$O(1)$ – konstantna složenost	Algoritmi koji se trenutno izvršavaju npr. koji izračunavaju matematičke funkcije, pristup elementu niza ili liste itd...
$O(n)$ – linearna složenost	Algoritmi kod kojih je potrebno proći kroz ceo niz, npr. održivanje max/min niza, učitavanje ili ispis svih elemenata niza ili liste
$O(n^2)$ – kvadratna složenost	Algoritmi sa ugnežđenim petljama, algoritmi za sortiranje selekcijom ili umetanjem
$O(n^3)$ – kubna složenost	Algoritmi sa višestruko ugnežđenim petljama, algoritmi za množenje matrica
$O(\log(n))$ – logaritamska složenost	Izuzetno efikasni, algoritmi binarne pretrage
$O(n\log(n))$ – log-linearna složenost	Efikan algoritam zasnovan na dekompoziciji (npr. sortiranje objedinjavanjem), korišćenju struktura podataka sa log vremnom pristupa, najpoznatiji Merge sort i Heap sort algoritmi
$O(2^n)$ – eksponencijalna složenost	Izuzetno neefikasni, algoritmi ispitivanja svih podskupova
$O(n!)$ – faktorijel složenost	Izuzetno neefikasni, algoritmi ispitivanja svih permutacija

Θ i Ω - notacije

- Θ i Ω notacije su takođe asimptotske notacije
- Kao što je O-notacija analogna relaciji $\leq$ za funkcije, definišu se analogni zapisi za relacije $\geq$ i $=$

Θ -notacija: Ako postoje pozitivne konstante c_1 i c_2 i prirodan broj N takvi da za funkcije f i g nad prirodnim brojevima važi $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ za sve brojeve $n > N$, onda važi $f(n) = \Theta(g(n))$ i čitamo „ f je veliko ‘teta’ od g “

- Θ -notacija pokazuje da funkcija $g(n)$ predstavlja asimptotski najbolju ocenu za funkciju $f(n)$

Ω -notacija: Ako postoje pozitivna realna konstanta c i prirodan broj N takvi da za funkcije f i g nad prirodnim brojevima važi $f(n) \geq c \cdot g(n)$ za sve prirodne brojeve $n > N$ onda je $f = \Omega(g)$ i čitamo „ f je veliko ,omega' od g ,“

- Ω -notacija pokazuje da funkcija $g(n)$ predstavlja asimptotski donju granicu za funkciju $f(n)$

Θ-notacija

- "Veliko Θ" određuje donju i gornju granicu rasta neke funkcije
- Primer: $f(x) = 3x^2 - 20x - 1$ $3x^2 - 20x - 1 = \theta(x^2)$

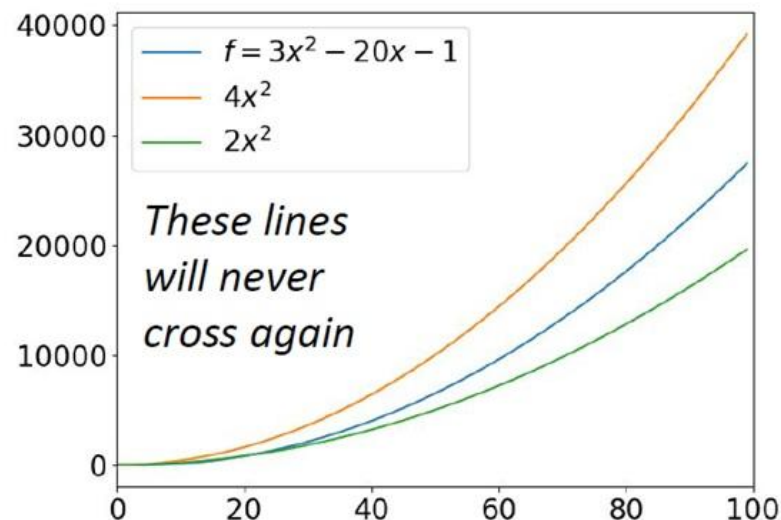
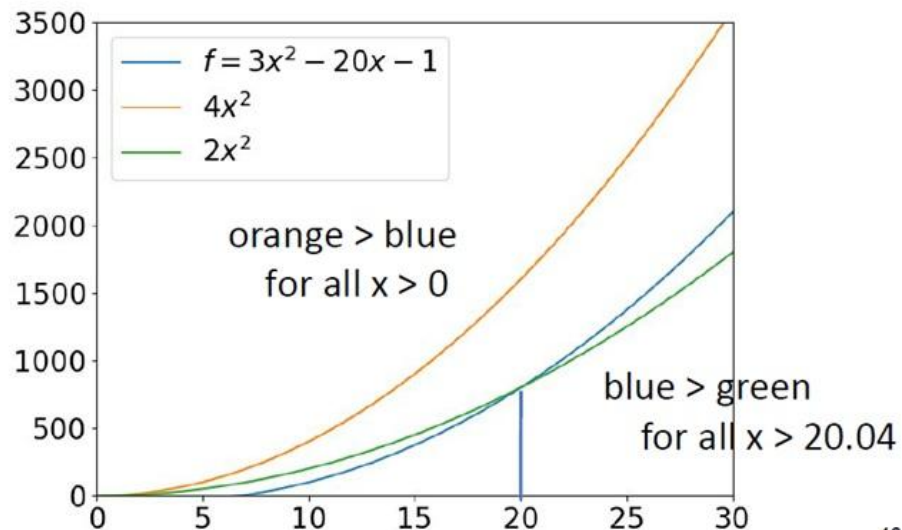
$f(x) = \Theta(g(x))$ means:

there exist constants c_0, x_0 for which $c_0 g(x) \geq f(x)$ for all $x > x_0$

and constants c_1, x_1 for which $c_1 g(x) \leq f(x)$ for all $x > x_1$

- Example, $f(x) = \Theta(x^2)$ because $4x^2 > 3x^2 - 20x - 1 \quad \forall x \geq 0 \quad (c_0 = 4, x_0 = 0)$
and $2x^2 < 3x^2 - 20x - 1 \quad \forall x \geq 21 \quad (c_1 = 2, x_1 = 20.04)$

Big O definition



$0 \leq x \leq 100$

Vreme izvršavanja i klase složenosti

- Neka je funkcija $f(n)$ jednaka broju instrukcija koje zadati algoritam izvrši za ulaz veličine n (logaritamska, linearna, log-linearna, kvadratna, eksponencijalna i faktorijel)
- Tabela prikazuje potrebno vreme izvršavanja algoritma ako se pretpostavi da je za izvršavanje jedna instrukcije potrebno vreme od 1ns

$n \setminus f(n)$	$\log n$	n	$n \log n$	n^2	2^n	$n!$
10	$0.003 \mu s$	$0.01 \mu s$	$0.033 \mu s$	$0.1 \mu s$	$1 \mu s$	$3.63 ms$
20	$0.004 \mu s$	$0.02 \mu s$	$0.086 \mu s$	$0.4 \mu s$	$1 ms$	$77.1 god$
30	$0.005 \mu s$	$0.03 \mu s$	$0.147 \mu s$	$0.9 \mu s$	$1 s$	$8.4 \times 10^{15} god$
40	$0.005 \mu s$	$0.04 \mu s$	$0.213 \mu s$	$1.6 \mu s$	$18.3 min$	
50	$0.006 \mu s$	$0.05 \mu s$	$0.282 \mu s$	$2.5 \mu s$	$13 dan$	
100	$0.007 \mu s$	$0.1 \mu s$	$0.644 \mu s$	$10 \mu s$	$4 \times 10^{13} god$	
1,000	$0.010 \mu s$	$1.0 \mu s$	$9.966 \mu s$	$1 ms$		
10,000	$0.013 \mu s$	$10 \mu s$	$130 \mu s$	$100 ms$		
100,000	$0.017 \mu s$	$0.10 \mu s$	$1.67 ms$	$10 s$		
1,000,000	$0.020 \mu s$	$1 ms$	$19.93 ms$	$16.7 min$		
10,000,000	$0.023 \mu s$	$0.01 s$	$0.23 s$	$1.16 dan$		
100,000,000	$0.027 \mu s$	$0.10 s$	$2.66 s$	$115.7 dan$		
1,000,000,000	$0.030 \mu s$	$1 s$	$29.9 s$	$31.7 god$		

Vreme izvršavanja i klase složenosti

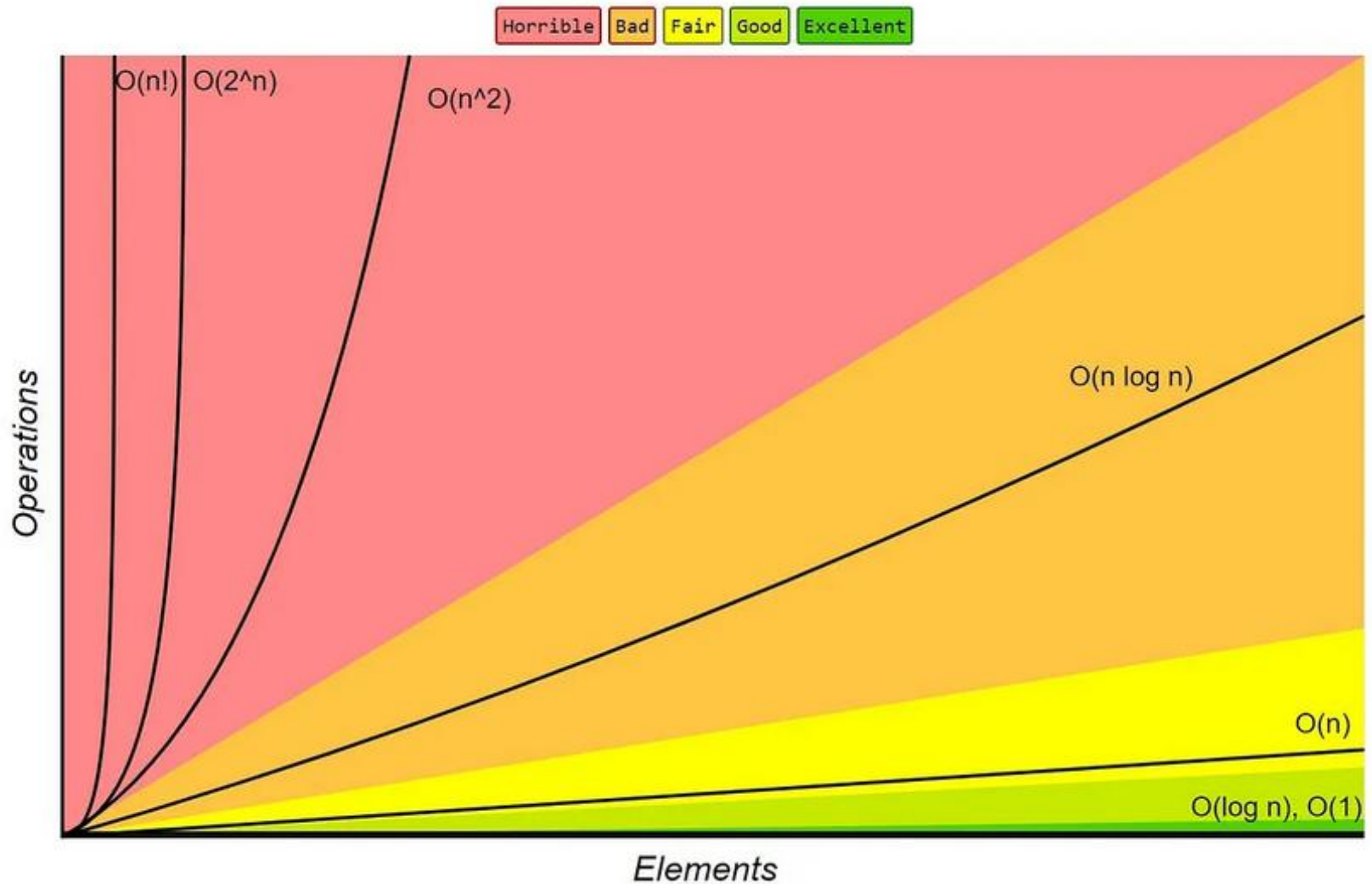
- Algoritmi čija je složenost **odozgo ograničena** polinom-funkcijama generalno su **efikasni**
- Algoritmi čija je složenost **odozdo ograničena** eksponencijalnom ili faktorijel funkcijom se smatraju **neefikasnim**
- Vodeći član funkcije $f(n)$ određuje potrebno vreme izvršavanja algoritma
- Primer iz tabele: Za broj instrukcija n^2+2n i ulaz veličine 1.000.000, član n^2 uzima **16.7 min.** dok član $2n$ uzima samo **2 μ s**
- Za $f(n)=2n^2+3n+2$ vreme od faktora $3n$ (**3ms**) i od faktora 2 (**2ns**) će biti zanemarljivo u odnosu na vreme od faktora $2n^2$ (**~33 min**)
- Vremenska (prostorna) složenost je uglavnom određena „vodećim” članom u izrazu koji određuje broj potrebnih instrukcija
- Na upotrebljivost algoritma ne utiču mnogo multiplikativne i aditivne konstante, koliko asimptotsko ponašanje broja instrukcija u zavisnosti od veličine ulaza

Složenost funkcije rasta



CLASS	n=10	= 100	= 1000	= 1000000
O(1)	1	1	1	1
O(log n)	1	2	3	6
O(n)	10	100	1000	1000000
O(n log n)	10	200	3000	6000000
O(n^2)	100	10000	1000000	1000000000000
O(2^n)	1024	12676506 00228229 40149670 3205376	1071508607186267320948425049060 0018105614048117055336074437503 8837035105112493612249319837881 5695858127594672917553146825187 1452856923140435984577574698574 8039345677748242309854210746050 6237114187795418215304647498358 1941267398767559165543946077062 9145711964776865421676604298316 52624386837205668069376	Good luck!!

Klase složenosti



Konstantna složenost $O(1)$

- Za algoritme klase $O(1)$ svojstveno je to da vreme izvršavanja i broj operacija koje je potrebno obaviti ne zavise od veličine ulaznih podataka, potrebno je pristupiti samo jednom elementu, i uvek se obavlja isti broj operacija
- Primer $O(1)$ je pristup elementu statičkog niza ili liste, za očitavanje $a[0]$ (element na početku), udaljenog i elementa na kraju potrebno je isto vreme.
- Dodavanje elemenata na stek i uklanjanje elemenata sa steka takođe su algoritmi složenosti $O(1)$

```
my_list = [1, 2, 3, 4, 5]
```

```
def get_first_element(arr):  
    return arr[0] # Accessing the element at index 0 takes constant time  
  
print(get_first_element(my_list)) #  $O(1)$ 
```

Linearna složenost $O(n)$

- Kada se mala količina obrade izvrši nad svakim ulaznim podatkom, vreme izvršavanja programa je linearno
- To je optimalno vreme izvršavanja za algoritam koji mora da obradi n ulaznih podataka ili da proizvede n izlaznih rezultata
- Algoritmi linearne složenosti samo jednom pristupaju svakom elementu ulaza (broj operacija koje je potrebno obaviti direktno je proporcionalan veličini ulaznih podataka)
- Primer: Ispis svih elemenata Python liste (iteracija kroz celu listu)

```
my_list = [1, 2, 3, 4, 5]
```

```
def print_all_elements(arr):  
    for element in arr: # We visit each element once  
        print(element)
```

```
print_all_elements(my_list) #  $O(n)$ 
```

Kvadratna složenost $O(n^2)$

- Kvadratno vreme izvršavanja nastaje za algoritme koji obrađuju sve parove ulaznih podataka (npr. u dvostruko ugneždenoj petlji)
- Algoritam ovog tipa treba primeniti samo za mali broj ulaza (malo n)
- Ovakvu složenost imaju elementarni metodi sortiranja, npr. selection sort ili bubble sort, kod kojih se elementi porede „svaki sa svakim“, i smatraju se uglavnom neefikasnim
- Primer: Poređenje parova elemenata Python liste (ugrađena petlja `for`)

```
my_list = [1, 2, 3, 4, 5]

def find_duplicate_pairs(arr):
    pairs = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)): # Nested loop
            if arr[i] == arr[j]:
                pairs.append((arr[i], arr[j]))
    return pairs

print(find_duplicate_pairs(my_list)) #  $O(n^2)$  in the worst case
```

Kubna složenost $O(n^3)$

- Kubno vreme izvršavanja nastaje za algoritme koji obrađuju sve trojke ulaznih podataka (najčešće u trostruko ugnežđenoj petlji)
- Algoritam $O(n^3)$ vremenom je primenljiv za mali broj ulaza (malo n)
- Kada je $n=100$, vreme izvršavanja je 10^6 , kada se n udvostruči, vreme izvršavanja se uveća 8 puta
- Primer: Množenje 2 kvadratne matrice $n \times n$, sa tri ugrađene petlje (iteracija kroz redove matrice A, kroz kolone B i kroz redove B ili kolone A)

```
# Perform matrix multiplication
for i in range(rows_a):          # Iterate through rows of matrix A
    for j in range(cols_b):      # Iterate through columns of matrix B
        for k in range(cols_a): # Iterate through rows of matrix B
            result_matrix[i][j] += matrix_a[i][k] * matrix_b[k][j]

return result_matrix
```

Logaritamska složenost $O(\log(n))$

- Za algoritme logaritamske složenosti vreme izvršavanja lagano raste
- Za $n=1000 \rightarrow \log n$ je 3, za $n=10^6 \rightarrow \log n$ je 6, tj. samo $2\times$ veći
- Za $n=1000$, $\log_2 n$ je ~ 10 , što je malo više nego za prethodni slučaj
- Bez obzira na osnovu, kada se n udvostruči, $\log n$ poraste za konstantu, a $\log n$ se udvostruči tek kad se n kvadrira
- Najpoznatiji primer je algoritam binarne pretrage, koji u nizu od 32 elementa pronalazi element u najviše 5 koraka, u nizu od 64 elemenata u najviše 6 koraka, ili u nizu od milion elemenata u najviše 20 koraka

```
def binary_search(arr, target):  
    left, right = 0, len(arr) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1
```

```
sorted_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]  
print(binary_search(sorted_list, 23)) #  $O(\log n)$ 
```

Primer: Algoritam binarne pretrage u sortiranoj listi

Umesto da proverava svaki element redom, binarna pretraga u svakom koraku deli skup na pola. Pretraživanje se vrši tako što se nađe srednji element skupa, proveriti je li jednak traženoj vrednosti, ako jeste vraća se njegov indeks, ako nije pretraga se nastavlja nad podskupom manjih (ako je srednji element veći od tražene vrednosti) ili većih elemenata (ako je srednji element manji).

Log-linearna složenost $O(n \log(n))$

- Klasa $O(n \log n)$ kombunuje log i linearne složenosti i čest je oblik algoritma
- $n \log n(n)$ složenost imaju algoritmi koji problem rešavaju tako što ga raščlane u manje, reše te potprobleme nezavisno jedan od drugog i kombinuju rešenja
- Ovakvu vremensku složenost imaju npr. **Heap** i **Merge sort** algoritmi sortiranja

```
import math

def primer_n_log_n(n):
    """
    Jednostavan primer koji ilustruje  $O(n \log n)$  složenost.
    """
    for i in range(n):
        # Logaritamska operacija (simulacija)
        broj_iteracija = 0
        k = n
        while k > 1:
            k //= 2
            broj_iteracija += 1
        print(f"Iteracija {i+1}: Broj logaritamskih koraka = {broj_iteracija}")

# Primer upotrebe:
velicina_ulaza = 10
primer_n_log_n(velicina_ulaza)

velicina_ulaza = 100
primer_n_log_n(velicina_ulaza)
```

Primer: Algoritam prikaza broja iteracija tokom sortiranja

-Spoljašnja petlja **for** se izvršava n puta, gde je n ulazna veličina (faktor n u složenosti)

-Unutrašnja **while** petlja u svakoj iteraciji spoljašnje petlje smanjuje vrednost k (inicijalno postavljenju na n) za polovinu u svakom koraku i izvršava se $\log_2(n)$ puta dok se ne dođe do 1

-Pošto se log operacija (simulirana unutrašnjom petljom) izvršava unutar linearne petlje (spoljašnja petlja), ukupna vremenska složenost ovog jednostavnog primera je $O(n \log n)$

Eksplozivna složenost $O(2^n)$

- Algoritmi sa eksplozivnim vremenom izvršavanja nastaju kod primene rešenja "grube sile" (*brute-force*)
- Vreme izvršavanja algoritma se drastično povećava sa malim povećanjem ulaza
- Ovakvi algoritmi su vrlo retko praktični za upotrebu
- Kada je npr. $n=20$, vreme izvršavanja je milion, kada se n udvostruči, vreme izvršavanja se kvadrira

```
def fibonacci_rekursivno(n):  
    """  
    Izračunava n-ti Fibonacciev broj koristeći čistu rekursiju.  
    Ovaj pristup ima eksplozivnu vremensku složenost  $O(2^n)$ .  
    """  
    if n <= 1:  
        return n  
    else:  
        return fibonacci_rekursivno(n - 1) + fibonacci_rekursivno(n - 2)  
  
# Primer upotrebe:  
if __name__ == "__main__":  
    velicine_ulaza = [5, 10, 15, 20, 25]  
    for n in velicina_ulaza:  
        print(f"Računanje {n}. Fibonaccievog broja...")  
        rezultat = fibonacci_rekursivno(n)  
        print(f"{n}. Fibonacciev broj je: {rezultat}")  
    # Obratite pažnju na vreme izvršavanja za veće vrednosti n!
```

Primer: Određivanje Fibonaccievog niza rekursivno

- Funkcija `fibonacci_rekursivno(n)` poziva samu sebe dva puta za svaku vrednost n veću od 1: jednom za $n-1$ i jednom za $n-2$
- Svaki od ovih poziva dalje generiše dva nova poziva, i tako dalje
- Broj rekursivnih poziva raste eksplozivno sa n , tj. sa 2^n
- Za $n=5$ imamo 15 poziva, za $n=10$, imamo oko 177 poziva, za $n=20$ preko 21.000 poziva, za $n=30$ preko 2.5 miliona poziva.

Faktorijel složenost $O(n!)$

- Algoritmi sa faktorijelnom složenošću su izuzetno spori i postaju nepraktični čak i za vrlo male ulazne veličine
- Sa svakim malim povećanjem veličine ulaza, vreme izvršavanja algoritma sa faktorijelnom složenošću eksponencijalno raste do tačke gde postaje praktično nemoguće izvršiti ga čak i za umereno velike ulaze
- U praksi, algoritmi sa faktorijelnom složenošću se obično izbegavaju osim za vrlo male ulazne skupove podataka

```
def heap_permutation(data, n):  
    if n == 1:  
        print(data)  
        return  
  
    for i in range(n):  
        heap_permutation(data, n - 1)  
        if n % 2 == 0:  
            data[i], data[n-1] = data[n-1], data[i]  
        else:  
            data[0], data[n-1] = data[n-1], data[0]  
  
if __name__ == '__main__':  
    data = [1, 2, 3]  
    heap_permutation(data, len(data))
```



[1, 2, 3]
[2, 1, 3]
[3, 1, 2]
[1, 3, 2]
[2, 3, 1]
[3, 2, 1]

2! = 2 x 1 = 2
3! = 3 x 2 x 1 = 6
4! = 4 x 3 x 2 x 1 = 24
5! = 5 x 4 x 3 x 2 x 1 = 120
6! = 6 x 5 x 4 x 3 x 2 x 1 = 720
7! = 7 x 6 x 5 x 4 x 3 x 2 x 1 = 5.040
8! = 8 x 7 x 6 x 5 x 4 x 3 x 2 x 1 = 40.320

Primer: Algoritam za određivanje svih permutacija u heap strukturi

Sa povećanjem broja ulaza, algoritam se izvršava po faktorijel principu

Klase složenosti struktura podataka

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Stack</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Queue</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Singly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Doubly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Skip List</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n \log(n))$
<u>Hash Table</u>	N/A	$\theta(1)$	$\theta(1)$	$\theta(1)$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Binary Search Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Cartesian Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>B-Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Red-Black Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Splay Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>AVL Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>KD Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$

Klase složenosti algoritama za sortiranje

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$